Mahitha Kalaga

Project Title

Video Demo: https://youtu.be/vQwIPN550TQ?feature=shared

---

1. Introduction

This project implements a secure multi-user blogging platform with role-based access control, CSRF/XSS/SQL-Injection protections, and real-time chat. All code is pushed to our private repository above. We leveraged PHP's password_hash, prepared statements, and defense-in-depth principles to achieve a robust system where superusers and regular users have distinct privileges.

---

2. Design

2.1 Database

- **users** (username PK, password_hash, role ENUM('user','super'), disabled TINYINT)
- **posts** (id PK, author FK→users.username, title, body, created_at, updated_at)
- **comments** (id PK, post_id FK→posts.id, author FK→users.username, body, created_at)

Relationships enforce referential integrity (ON DELETE CASCADE).

2.2 User Interface

- **ML5/CSS3** responsive forms with client-side validation (required, pattern, minlength)
- **Customized pages** for login, registration, dashboard, post creation/listing, comment CRUD, and user management
- **Secure cookie** flags (HttpOnly, Secure), session timeouts, and uniform header/footer

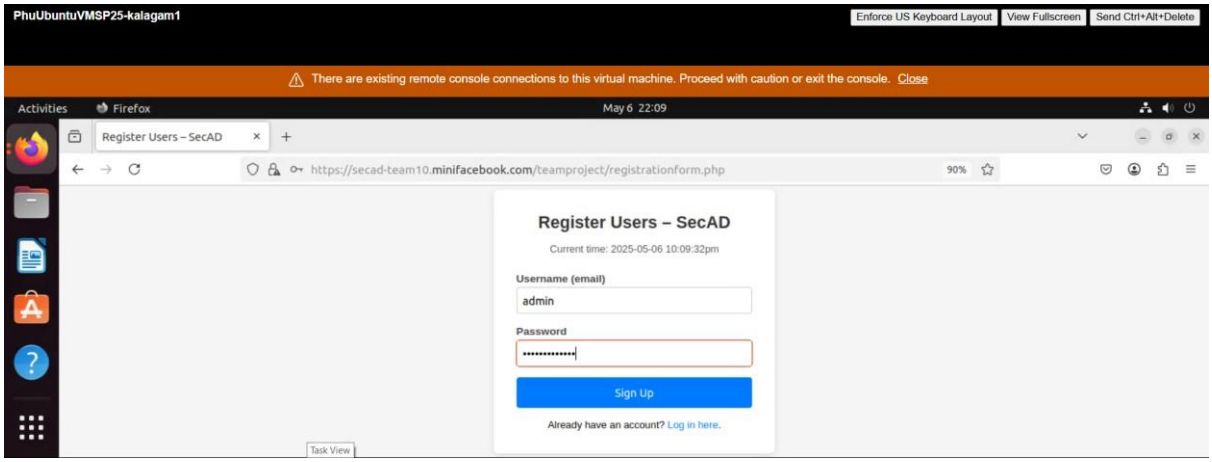
2.3 Features

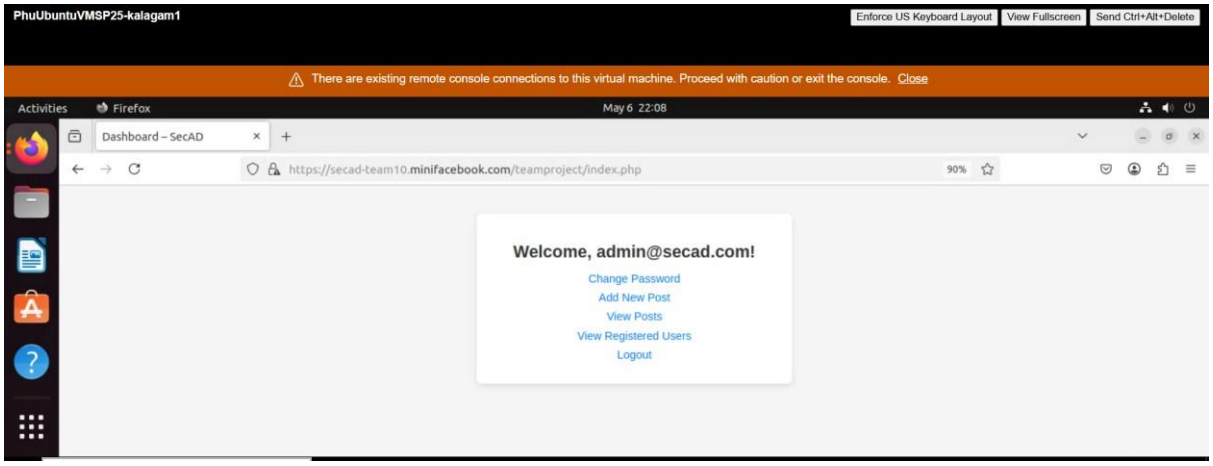
- **Standard users**
- Register, login, password change
- Create, read, edit, delete own posts
- Add, edit, delete own comments
- Real-time chat through WebSocket endpoint
- **Superusers
- Login only (no registration form)
- List all registered users in an admin table
- Deactivate/activate user accounts (disabled users can't log in)

3. Implementation & Security Analysis

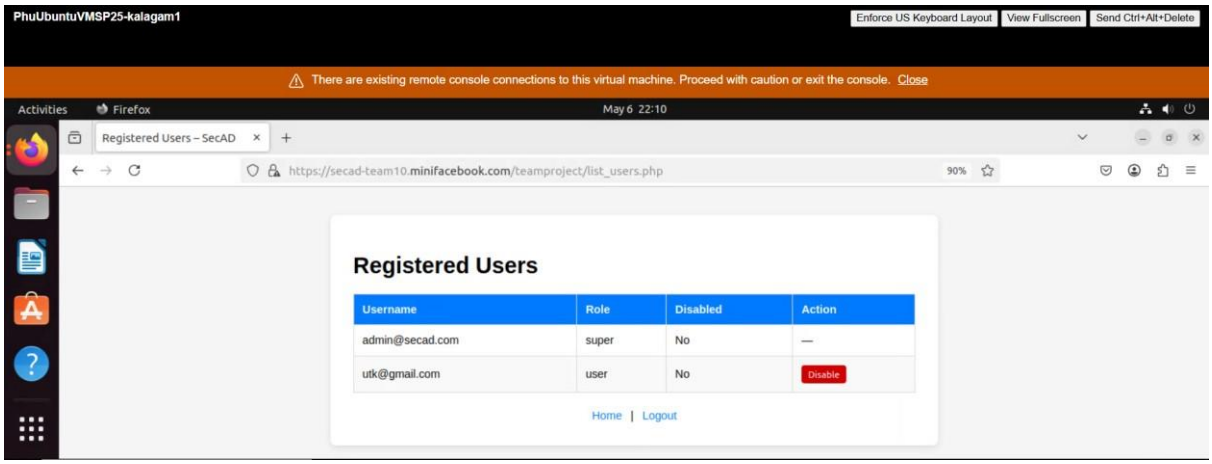
1. **Applied security principles** - Least privilege: non-root DB user, scoped prepared statements - Fail-safe defaults: disabled accounts locked at session guard
2. **Defense-in-depth & breadth** - Client & server input validation - Output encoding (htmlentities) - CSRF tokens on all state-changing forms - Session fixation & hijacking checks (User-Agent match, session_regenerate_id)
3. **Database security** - Prepared statements for all queries - Foreign keys for referential integrity - Secure hashing (password_hash) for new users
4. **Solid & defensive code** - One init.php for session & DB setup - Strict if checks on everything, with JS alerts + redirects - HTTP headers to force HTTPS (in production)
5. **Attack defenses -** XSS: **Sanitize all user-supplied data on output -** SQL Injection: **Never concatenate strings directly; always use prepared statements -** CSRF: **One-time tokens on each form, checked on every POST -** Session Hijacking:** Binding to User-Agent, secure flags on cookies, session expiration
6. **Role separation** - session_auth.php requires $_SESSION['role'] === 'super' for admin pages - UI components and hidden links for unauthorized roles

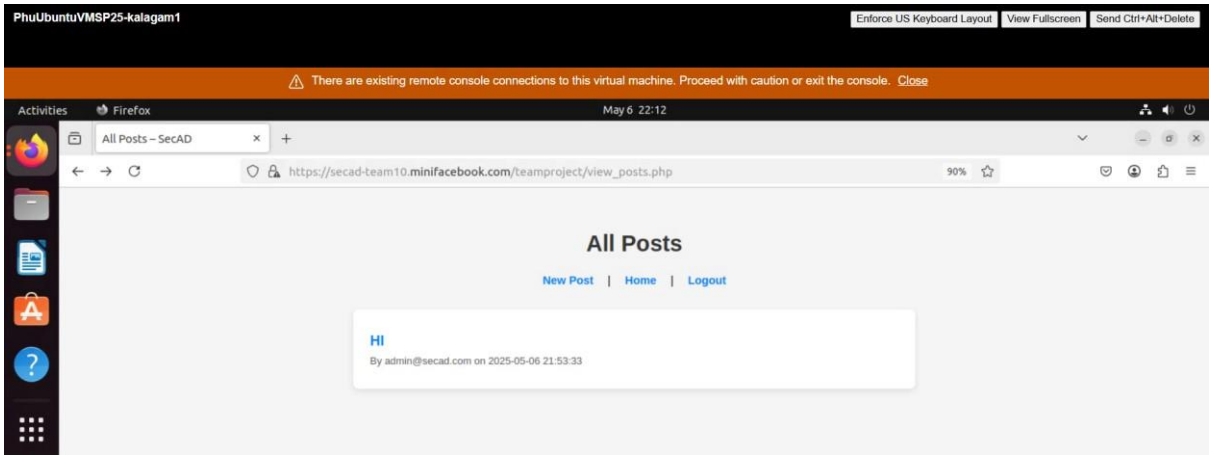
Registration & Login

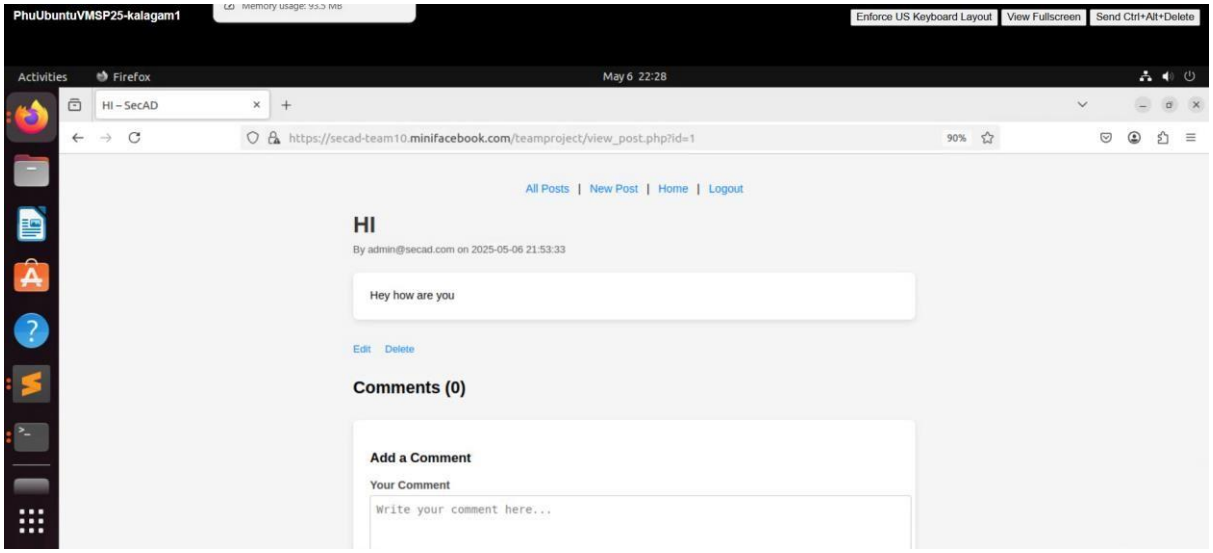




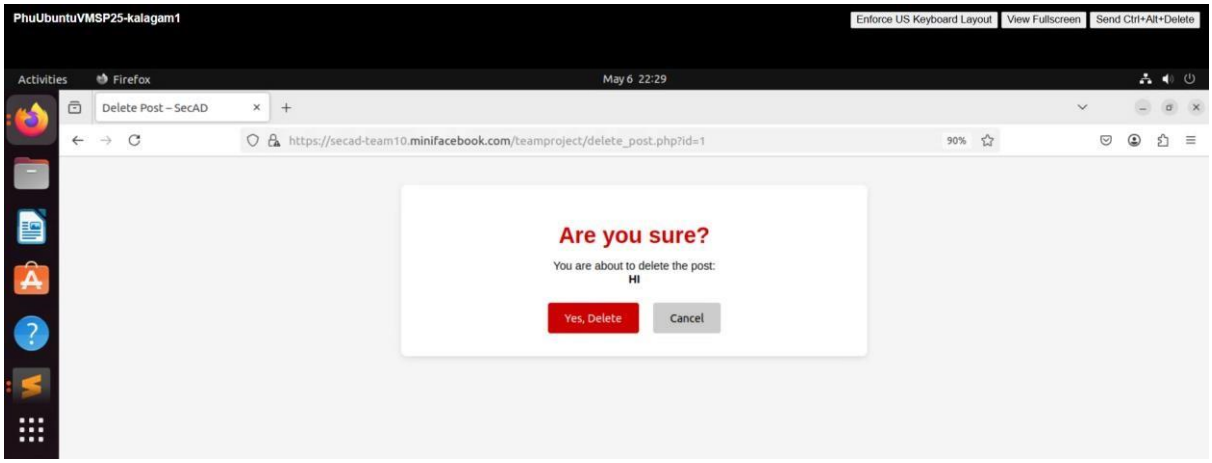
Superuser Enable/Disable

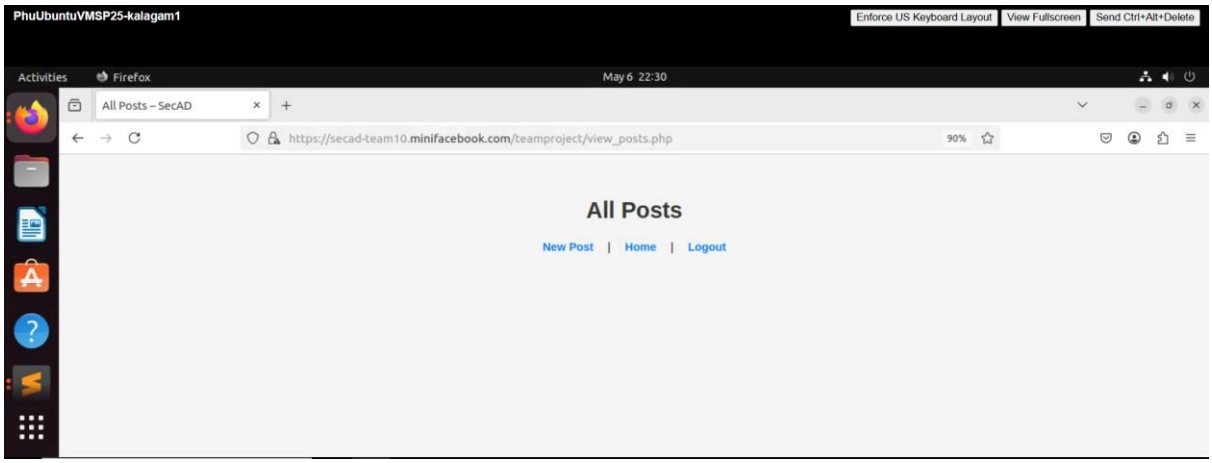




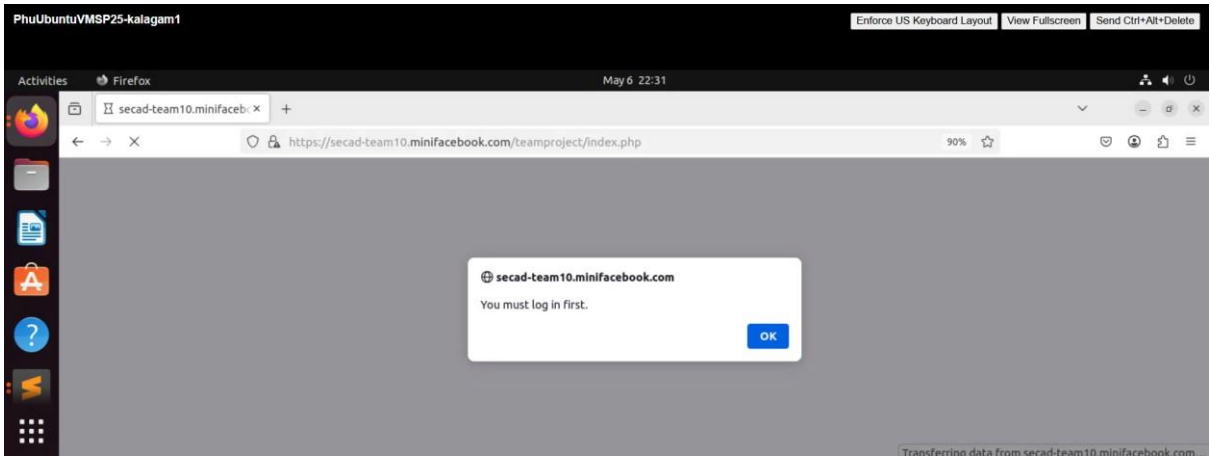


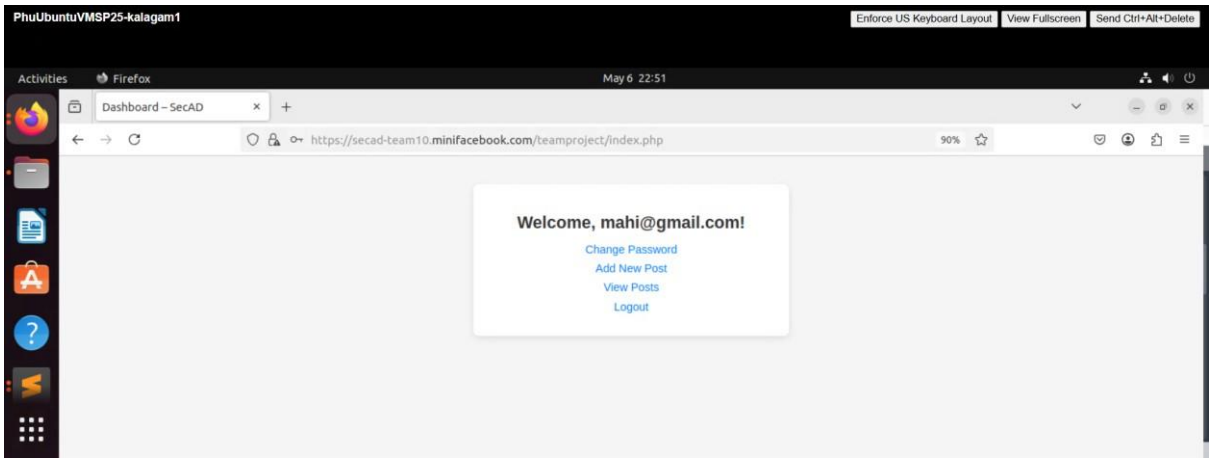
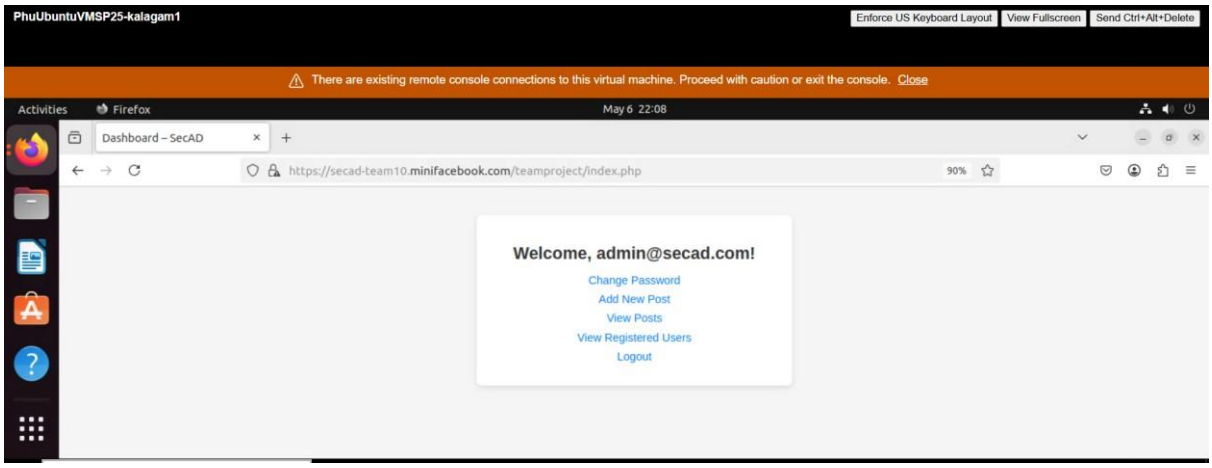
Post Deletion





CSRF Protection





Role-Based Access